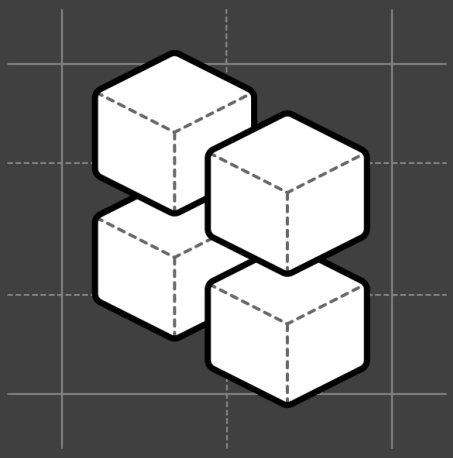
Level Design Add-on

Seventeen patterns that move activation, retention, and expansion, each built on a rule of game design.



What's inside:

- The level-design rule under each pattern, in plain language, so you know why it works and not just that it works.
- 17 patterns across activation, retention, and expansion, each with named examples from products you already use.
- The agent move for every pattern, plus how to run the whole file with your favorite LLM.

Contents

INTRODUCTION

The swipe file, not another protocol

START HERE

The other three protocols in this stack are written to read front to back, each one walking you through one level of the game: activation, retention, expansion. This one is different. This is the swipe file.

Seventeen patterns, each one a thing you can actually ship. You don't read it cover to cover. You find the place you're losing users, turn to the patterns for that stage, and build. It's the parts catalog that sits next to the three protocols.

What makes this the add-on and not a fourth protocol is the layer under each pattern. Every one of these seventeen is built on a rule from game design. Not gamification, not badges and points; the actual structural rules that let a well-made game take a total stranger and turn them into someone fluent, with no manual and no training. That is the exact problem onboarding has. Games solved it decades ago. So under each pattern you'll find the rule it stands on, in plain language, because once you see the rule you can invent your own patterns, not just copy mine.

One word I use throughout: the agent. I mean the AI built into your product, the one that can talk to the user and do things for them. It covers both kinds of product: the agent-led, AI-native ones where the AI is the whole interface, and the traditional products now bolting an AI layer on. Either way, the agent is the first time onboarding has had something that can watch what each user does and respond to that specific person. Every pattern here has an agent move, because most of them get sharper when the agent runs them. That's the bet behind this file.

> **Most products won't need all seventeen. The ones that move the number pick two or three and ship them well. Start where it hurts most.**

The lens everything runs on

The game: why game design is the lens

THE FLOW CHANNEL · THE LOOP · FEEDBACK

Games are the only discipline that reliably takes a confused stranger to mastery with no manual. Nobody reads the instructions for a video game. They press a button, something happens, the game teaches them the next thing exactly when they need it, and by the end they're fluent in a system they couldn't have described an hour earlier. That's onboarding. Games just have a fifty-year head start. Three ideas carry most of the weight in this file, so here they are up front.

The flow channel. A game has to keep you in a narrow band between bored and overwhelmed. Too easy and you drift off; too hard and you quit. The catch is the band keeps moving, because as your skill grows, yesterday's challenge becomes today's boredom, so the challenge has to keep rising, forever. This is the whole argument for why onboarding can't be a phase that ends at day seven. A product that teaches you everything in week one and then goes quiet has parked you in the boring half of the channel. That's the day-30 drop-off everyone files under retention. It's really a leveling problem.

The loop. Underneath the levels runs a small engine that repeats every session: a cue that points you at the next thing, an action you take, and a reward that makes the action feel worth it and points you at the next cue. Cue, action, reward. Good products run this loop tightly. Bad ones ask for actions and forget the reward, or reward things the user never cared about.

Feedback. Every meaningful action needs a visible answer: what happened, what it means, what it unlocked. A basketball net adds nothing to the rules of basketball; it exists so everyone can see the ball went in. Silent success reads as failure, and most of the friction in onboarding is just missing feedback.

> **Hold those three and the seventeen patterns stop looking like a list of tactics and start looking like a system. Each pattern is one of these ideas, made concrete for a stage and a moment.**

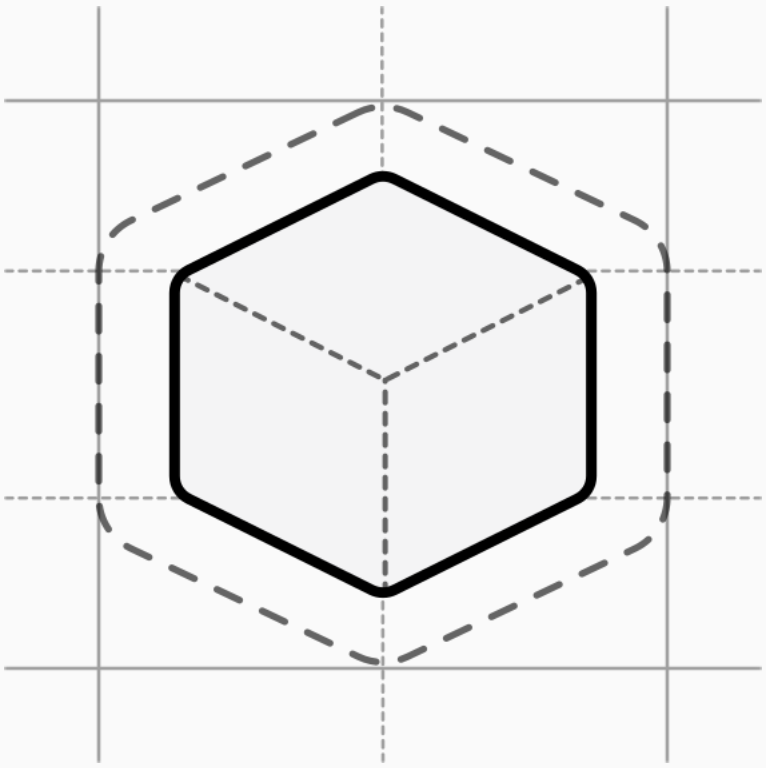
All 17 on one page

The rule each pattern stands on, and when to reach for it. The page number for each is in the contents.

#	PATTERN	THE RULE UNDER IT	REACH FOR IT WHEN
1	Aha moment shortcuts	A goal only pulls if it's wanted before it's reached	Users sign up and never come back
2	Contextual tooltips	Rules are discovered through play, not memorized	Your tour gets skipped
3	Personalized paths	The candy store: a curated few beats infinite choice	Different users need different first steps
4	Empty state design	Nobody quits a puzzle that arrived solved	A screen can show zero data
5	Onboarding checklists	A puzzle lets you feel yourself getting closer	Users start setup and stall halfway
6	Interactive product tours	What happens to you beats what happens to others	You're explaining instead of letting them do
7	Behavior-based triggers	Raise the challenge when skill makes the old level boring	Your emails fire on a clock, not a signal
8	Time-delayed nudges	The mentor notices you're stuck, then offers a hand	Users freeze on a screen and leave
9	Social proof in context	Players read the room and calibrate to it	Users hesitate at a decision point
10	Friction logging	The failed attempt is the curriculum	You know where users drop off, not why
11	Comparative benchmarks	People want to be judged, and judged fairly	Your dashboards show activity, not standing
12	Hotspots and pulsing dots	The interface is a promise about what's possible	You ship features nobody notices
13	Inline announcements	The change has to be visible where it happens	Your feature emails go unopened
14	The "What's New" widget	Always a next thing, and let them pull it	Engaged users want to keep up, on their terms
15	Usage-based suggestions	Serve the request and steer one level up	Everyone sees the same upsell
16	Win-back flows	The value you've built is the pull back	Users go quiet before they cancel
17	Usage milestones	Experiences are the only thing that change people	Users create value they never get to feel

Activation

From signup to first value. This is the front door, and most products lose more people here than anywhere else. The whole job of activation is getting one real win into the user's hands fast, before they decide nothing is happening.



Activation · Pattern

01

Aha moment shortcuts

THE RULE: A GOAL ONLY PULLS IF IT'S WANTED BEFORE IT'S REACHED

The principle. In game design there's a rule about goals: a goal only works if the player wants the reward before they reach it, not after. Chess works as a pitch in four words, capture the other king, and nobody explains checkmate first. The trouble with most onboarding is that the goal it sets, "complete your setup," is rewarding to the vendor and meaningless to the user. There's a second rule stacked on top, from puzzle design: the pleasure of "aha" comes from seeing the answer, not from grinding toward it. Nobody abandons a Rubik's Cube as impossible, because it ships solved; you've seen the answer, so you believe the journey is survivable.

In the product. The flows that convert are built around getting the user to one specific moment of value fast, not around teaching the product. Growth circles call it the aha moment. For Slack it was sending 2,000 messages; for Dropbox, uploading the first file; for Facebook, seven friends in ten days. Hit that threshold and retention bends permanently upward. The mechanism is the peak-end rule: people judge an experience by its most intense moment and how it ended. If your most intense moment is a twelve-screen setup wizard, you have a problem. If it's a small win in the first ninety seconds, you have a product. A Rocketlane survey in 2025 found 83% of B2B buyers call slow onboarding a dealbreaker, and slow really means "I haven't felt anything yet." Trello drops you straight into a demo board with cards already on it, so the first action is using the product, not configuring it.

The agent move. An agent can build the solved cube live and personal. "Here's a mock version of the dashboard you could have by Friday, built from your industry. Want me to start on the real one?" The preview is generated, specific to them, and disposable, which is something a static product could never afford to ship. It collapses the distance between signup and the wanted goal to a single exchange.

WHAT WORKS

- Pre-populate accounts with realistic sample content so the first action is use, not setup.
- Show the value before signup, so the wanted goal is established before the work starts.
- Engineer the first session to end on a win, not on a checklist.

WATCH FOR

- Onboarding that introduces every feature. Most features can wait until session two.
- Requiring a data import before first value. People drop off during the CSV upload.
- Hiding the aha moment behind a paywall or a "talk to sales" gate.

> **The shift. Onboarding isn't a tour of your product, it's the first session of using it. Treat session one as the demo, not the thing that comes before the demo.**

02

Contextual tooltips

THE RULE: RULES ARE DISCOVERED THROUGH PLAY, NOT MEMORIZED UPFRONT

The principle. Games gave up on written rulebooks a long time ago, because it's nearly impossible to encode how a system actually feels into a document nobody reads. They moved to teaching through play: you learn the rule at the exact moment you need it, by doing the thing. A product tour is the rulebook. It teaches you twelve things two minutes before any of them matter, which is to say it teaches you nothing that sticks.

In the product. The fix for a tour that gets skipped isn't a shorter tour, it's deleting the tour. The pattern that works is the contextual tooltip: one short hint at the precise moment someone first tries to use a feature. A Figma overlay when you first land on the canvas. A Slack tip when you create your first channel. Same lesson, but it arrives when it's relevant instead of before it's relevant. There's a name for why this beats the upfront slideshow, the spacing effect: people retain what they learn at the moment of need. In a Jimo case study, Genially switched from email feature announcements to in-app contextual tooltips and saw activation rise 25% with no change to the product underneath.

The agent move. A tooltip is a hint someone wrote in advance for a spot they guessed you'd get stuck. An agent doesn't have to guess. It can watch the actual action, see the hesitation, and explain the thing the user is actually reaching for, in their words, the first time they reach for it. The hint stops being pre-written furniture and becomes a real answer to a real moment.

WHAT WORKS

- Trigger on hover or on the attempt, not on a click the user has to go find. Click-triggered tips get ignored.
- Answer "what is this and why should I care," not just "what is this."
- Disclose progressively: the basic tip first, the pro tip later, the shortcut later still.

WATCH FOR

- More than two tooltips firing in one session. Users tune the whole system out.
- Tips on features the user hasn't even looked at. Just-in-time means in time, not just in case.
- Generic descriptions that read like a help article instead of a hint.

> **The shift. A tour teaches people what your product does. A tooltip teaches people what they're already trying to do. Only one of those sticks.**

03

Personalized onboarding paths

THE RULE: THE CANDY STORE, A CURATED FEW BEATS INFINITE CHOICE

The principle. Jesse Schell tells a story about working a candy store with sixty flavors. Recite all sixty and the customer panics somewhere around flavor 32. Say "name any flavor you want" and they freeze and walk out. But "we have just about everything, and our most popular are these six" sold candy all day. People want the feeling of freedom plus a small set of attractive choices. You don't have to give true freedom, only the feeling of it, shaped to who's standing in front of you.

In the product. Most onboarding treats every user the same way, and the data on that is grim: average SaaS activation sits around 30%. The fix is to ask two or three questions at signup and route each person through a different first five minutes based on the answers. Canva asks "what will you be using this for" and reshapes the entire empty state. Mailchimp asks about your goals and recommends specific first actions. Same product, different doorway. The mechanism is the cocktail party effect: people pay attention to what feels personally relevant, and a demo that mentions their role lands harder than one built for everyone. Moxo's 2025 report put the lift at 40% on retention versus a generic flow; Clevry put time-to-productivity at 52% faster.

The agent move. This is the candy clerk's whole job, and the agent is built for it. It can ask what the user came to do, then curate the menu to that answer, live, instead of routing them into one of three hardcoded buckets. "For someone in your role, people usually start with these two things." The feeling of freedom, the curation tuned per person, no twelve-branch flowchart to maintain.

WHAT WORKS

- Two or three questions, no more. Every extra field costs roughly 7% in conversion.
- Two to four distinct paths, not twenty. The goal is differentiation, not a custom build per user.
- Make the difference visible: different empty state, different first task, different welcome per path.

WATCH FOR

- Building twelve personas and one onboarding flow. The personas exist on paper, not in the product.
- Long questionnaires before the user has felt anything. You haven't earned the questions yet.
- Storing the answers and never changing the interface based on them.

> **The shift.** Your overall activation rate is a fiction. The real numbers are the per-segment ones, and most products are quietly leaving a fifth of their activation on the table by showing everyone the same default.

04

Empty state design

THE RULE: NOBODY QUITS A PUZZLE THAT ARRIVED SOLVED

The principle. Doubt about whether a thing is even solvable is what kills persistence. The reason nobody throws a Rubik's Cube across the room as impossible is that it arrived solved; you scrambled it yourself, so you know a solution exists. Show people what "solved" looks like and the work in front of them feels survivable. Hide it and an empty starting point reads as broken even when it's working exactly as designed.

In the product. The most dangerous screen in your product is the one right after signup: no data, no content, no idea what to do. Wyzowl found 80% of users have deleted an app because they didn't understand how to use it, and in my experience that confusion almost always traces to one screen, the empty dashboard. Call it blank page paralysis, the same reason a blank document is harder than editing a draft. Airtable is the canonical fix: new users never see a blank spreadsheet, they see a row of templates with sample data already in them, and "start from scratch" is there but it isn't the default. Most people pick a template, see structure, and feel productive within thirty seconds.

The agent move. A template is a solved cube someone built for the average user. An agent can hand the user a solved cube built from their world: pull one real source, generate a populated first view, and say "this is roughly what yours will look like; want me to make it real?" The empty state stops being a generic illustration with a button and becomes a preview of their own success.

WHAT WORKS

- Audit every screen that can show zero data. Most products have eight to twelve of them, not one.
- Put a clear next action on every empty state, not just the dashboard. Sub-features need it too.
- Offer two doors, template or blank, and default to the template.

WATCH FOR

- Empty states with motivational copy and no action. "Start your journey" helps no one start anything.
- Sample data the user can't delete. The product feels haunted.
- Empty states that look like error states. Users assume something broke.

> **The shift. An illustration plus a real next action feels intentional. A blank table feels like the product is broken. The gap between those two is one design pass and a few hundred words.**

05

Onboarding checklists

THE RULE: A PUZZLE LETS YOU FEEL YOURSELF GETTING CLOSER

The principle. A riddle either cracks or it doesn't, but a good puzzle lets you feel yourself getting closer, and that feeling of closing the gap is what keeps hands on it. Game designers stack small goals into medium ones into one clear peak, so you can always see both the next step and the summit. A checklist is that structure, borrowed: a visible arc that turns "explore the product" into "four specific things, and you can see how many are left."

In the product. Most checklists fail the same way tours do, built for the team shipping them instead of the user using them. The version that works is short, visible, and has the first item pre-checked. Sked Social documented a 3x lift in conversion after moving to a four-item checklist with the first task already complete, account creation. Pre-checking sounds like a trick, but it taps the endowed progress effect: people are more likely to finish something they've already started. There are actually three forces stacked here. The Zeigarnik effect, where unfinished tasks nag at us. Goal clarity, where a small finite list beats a vague "go explore." And endowed progress on top.

The agent move. A static checklist is the same five boxes for everyone, and it keeps showing you boxes you've already effectively done. An agent can check items off as it watches them happen, reorder the list around what this user actually needs next, and quietly retire the whole thing the moment they're activated, so it never curdles into wallpaper.

WHAT WORKS

- Four to six items only. The four-step checklists have the highest completion rates.
- Keep it visible for the first couple of weeks, in a sidebar or progress widget, not a popup.
- Make each item a link straight to the action itself. The checklist is a navigation surface, not a list.

WATCH FOR

- Twelve-item checklists. Users see the length and bail before starting.
- Items that don't link anywhere. "Configure your settings" with no link is just nagging.
- Checklists that stay up forever, even after completion. They become chrome.

> **The shift. A checklist isn't a to-do list, it's a contract about what activation looks like. Four specific completions and they get value. Twelve vague ones and they get fatigue.**

06

Interactive product tours

THE RULE: WHAT HAPPENS TO YOU BEATS WHAT HAPPENS TO OTHERS

The principle. Events that happen to us are simply more interesting than events that happen to someone else. Tetris grips people with zero story because everything in it is happening to you, right now, by your hand. There's a learning version of this called the generation effect: you remember what you generate far better than what you're shown, which is why flashcards beat re-reading and a sandbox beats a screenshot. A static tour is something happening to someone else while you watch. An interactive one is something happening to you.

In the product. A static tour is the "click Next to learn about projects" slideshow. An interactive tour skips the narration and just has you do the action: create the project, name it, watch it appear. Same lesson, but you're using the product instead of watching a demo of it. In a Jimo report across about a thousand tours, median completion was 15%; the interactive ones hit 44%, roughly 3x. The trick that protects the effect: let people keep what they made. Nothing kills momentum like watching your first real work vanish into a tutorial sandbox.

The agent move. The best interactive tour isn't a scripted "create a project called Test," which feels like homework. An agent can run the first quest on the user's actual stuff: connect one real source, produce one real artifact from it, and narrate what just happened. The user isn't doing a drill on fake data, they're doing the real job for the first time, with a companion. That's a story they were in, not a tutorial they sat through.

WHAT WORKS

- Three to five steps, no more. After that, completion falls off a cliff.
- Real actions on real data. Placeholder data breaks the spell instantly.
- Let people keep what they built during the tour.

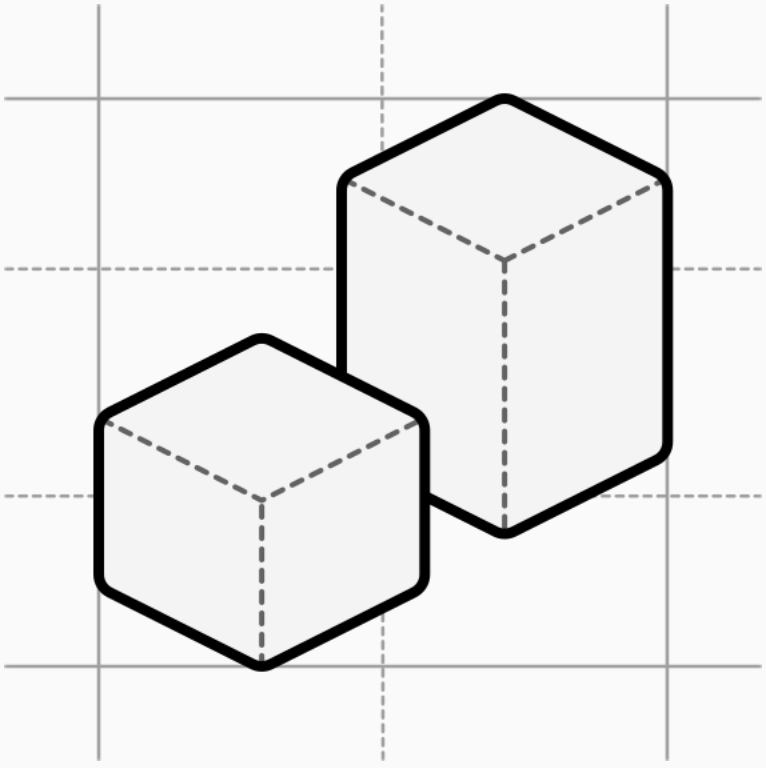
WATCH FOR

- Tours longer than seven steps. People bail around step four.
- Generic placeholder data. It reads as homework.
- Forcing the tour on everyone, including the power user opening their second account.

> **The shift. A tour isn't how you teach people your product, it's how you get them to their first real win. If they aren't doing anything in there, they aren't learning. They're sitting through a demo with extra steps.**

Retention

The right message at the right time. Retention is where the flow channel does its work: the challenge has to keep rising as the user's skill grows, or they drift off bored somewhere around day thirty. Every pattern here is about reading the moment and responding to it, instead of running a clock.



Retention · Pattern

07

Behavior-based triggers

THE RULE: RAISE THE CHALLENGE THE MOMENT SKILL MAKES THE OLD LEVEL BORING

The principle. The flow channel moves. As a player's skill grows, the level that was challenging last week becomes boring this week, and the game has to raise the difficulty exactly then, not on a schedule. The best games read where you actually are, what you've mastered and what you keep repeating, and meet you there. A retention email that fires on day three regardless of what you've done is the opposite: a difficulty curve set by a calendar, talking to a player it can't see.

In the product. Time-based onboarding emails fire on a clock, not a context. Day 3 reminder, day 7 nudge, day 14 check-in, all landing whether or not the user has done anything since signup. Behavior-based triggers fire when a specific action happens, or pointedly doesn't, inside the product. Loom prompts you to share your first video the moment you finish recording one. Grammarly waits until you've written a sentence with a clarity issue, then surfaces the upgrade. There's a Greek word for this, kairos, the opportune moment, and Jimo puts the lift from a behavior-triggered message at roughly tenfold over the same message on a clock.

The agent move. This is the agent doing what every good game does: it watches the signal and raises the next thing exactly when the user is ready for it. "You've run this report manually three weeks running. Want to learn the automation?" That lands because it arrives at the moment the old level got boring. The agent is the only part of your stack that can see that moment for each user.

WHAT WORKS

- Map your three to five highest-value events, like visiting pricing three times or inviting a teammate, and write one specific message per event.
- Cap triggered messages at one per session. More than that reads as the product harassing the user.
- Always include a dismiss. A triggered message with no off switch feels hostile.

WATCH FOR

- Triggering off vague events like "logged in five times." That's a usage threshold dressed up as behavior.
- Sending the same triggered message every time the event fires. Frequency caps matter.
- Nagging users to do something the product has already done for them.

> **The shift.** Drip campaigns send messages on a schedule. Behavior-based triggers send messages on a signal. The signal is always more interesting than the schedule.

Retention · Pattern

08

Time-delayed nudges

THE RULE: THE MENTOR NOTICES YOU'RE STUCK, THEN OFFERS A HAND

The principle. In the hero's journey the mentor equips the hero and then stays out of the fight; the point is that the hero's hands stay on the outcome. But a good mentor is also watching, and notices when you've gone quiet in a way that means stuck, not thinking. The skill is reading silence. A player frozen on a screen isn't reading, they're lost, and the mentor who catches that and offers one specific hand, without grabbing the controls, is the one who keeps them in the game.

In the product. Behavior triggers fire when something happens. Sometimes the most important signal is when nothing happens. A user who's been on the same screen for two minutes without a click isn't reading, they're stuck. The right move is a small, specific nudge, not "need help?" but "Having trouble connecting your CRM? Here's how to connect Salesforce." The mechanism is loss aversion used kindly: they've already spent time on this page, and a relevant nudge makes walking away feel more wasteful than pushing through. The Room, a hiring platform, added a 90-second inactivity nudge to their CV upload flow and saw uploads climb 75% in ten days, from around 210 a week to 350, with the nudge as the only change.

The agent move. A timed nudge is a guess about where people get stuck. An agent can tell stuck from thinking by what's actually on the screen, then offer the one specific thing this user needs, and then step back and let them do it. That's the mentor posture exactly: notice, equip, defer. Do it for them once, do it with them next, watch them do it after.

WHAT WORKS

- Wait 60 to 120 seconds. Earlier and you interrupt people who are still reading.
- Make the nudge specific to the page they're on. Generic "need help" widgets get ignored.
- Show it once per page per session.

WATCH FOR

- Firing the same nudge on every page. It reads as a chatbot with nothing to say.
- Linking to a 40-article help center instead of the one article that matters here.
- Nudging veterans past their first couple of weeks. They don't need it and find it patronizing.

> **The shift. Silence is data. A user who hasn't clicked in two minutes is telling you something. The product gets to decide whether to listen.**

09

Social proof in context

THE RULE: PLAYERS READ THE ROOM AND CALIBRATE TO IT

The principle. This one is a looser fit to a single game-design lens, so I'll be honest about it: the closest parallel is how a multiplayer world surrounds you with other players' visible activity, and you calibrate to the room without anyone instructing you. When you're unsure what to do, you look at what people like you are doing. Game designers lean on this constantly, shaping behavior through what's visibly happening around you rather than through a rule. The closer the comparison is to you, the more it moves you.

In the product. Most products treat social proof as a homepage thing, logos and big numbers and testimonials, and then the proof vanishes the second the user is inside. The version that actually moves usage is inline, shown at decision points within the product. Slack puts "X people from your company are already here" right next to the join button. Frase shows "join 30,000 content teams" inline with the feature it applies to. The mechanism is informational social influence, and the specificity is everything: "12 people from your company" beats "millions of users worldwide" every time, because it's the room you're actually in.

The agent move. Static inline proof still shows everyone the same line. An agent knows who this user is and which decision they're sitting on, so it can surface the proof that's actually relevant to this person at this fork, the room that looks like them, at the moment they're deciding whether to act or back out.

WHAT WORKS

- Place proof at decision points, sign up, invite, upgrade, not in a random sidebar.
- Make it specific. "12 people from your company" beats "millions worldwide."
- Show activity, not just totals. "3 teammates joined this week" beats "30,000 users."

WATCH FOR

- Numbers that obviously include dead accounts. Users can smell it.
- Irrelevant proof. A B2B PM doesn't care that consumer hobbyists love your tool.
- Proof shoved into a modal that interrupts. It should be ambient, not in the way.

> **The shift. A homepage logo wall is for convincing buyers. Inline social proof is for convincing users. They're different jobs, and most products only do the first one.**

10

Friction logging

THE RULE: THE FAILED ATTEMPT IS THE CURRICULUM

The principle. Simulations teach because they grant permission to fail and then let the learner see why the failure happened. A flight simulator is valuable precisely because crashing in one is educational instead of fatal. The failure is the lesson, but only if you catch it in the moment, with the context still attached. Wait until later and all you have is the wreckage and a guess.

In the product. Most teams know roughly where users drop off and almost never know why, because nobody asks while it's happening. Quarterly NPS is too late; exit interviews are too late; by the time anyone fills out a form, the user has already rationalized their way out. A 2024 Clevry study found only 12% of users rate their onboarding as effective, and most companies can't say why because they never asked during the experience. The fix is the contextual microsurvey: one question, fired the instant friction shows up, dismissible. Grammarly does it on the analytics page; Superhuman does it at key feature moments. What comes back isn't just a sentiment score, it's the narrative, "I couldn't figure out how to connect my calendar," which goes straight into the backlog.

The agent move. A microsurvey is you asking the user to stop and describe their own failure. An agent was already watching it happen. It can see the failed attempt, name the likely cause, and turn the moment into the lesson in real time, "that didn't work because the date field isn't mapped, here's what I'd try," while also logging the friction for you. Failure stops being a support ticket and becomes the curriculum, for the user and for your roadmap at once.

WHAT WORKS

- One question per survey. Two cuts response rates in half.
- Trigger on the friction event, not on a 30-day mark.
- Act on the answers within a week. The half-life of this feedback is short.

WATCH FOR

- Five-question forms dressed up as "quick surveys." Users notice and skip.
- Surveys that only fire for power users. The people you most need to hear from are already half gone.
- Collecting answers and never closing the loop. People stop responding the moment they sense nothing changes.

> **The shift. Usage data tells you where users dropped off. Asking in the moment tells you why. Both matter, and most teams only build the first one.**

11

Comparative benchmarks

THE RULE: PEOPLE WANT TO BE JUDGED, AND JUDGED FAIRLY

The principle. One need is common to almost everyone: the need to be judged. People don't hate being judged, they hate being judged unfairly. Games are beloved partly because they're excellent, objective, fair judges, and a player who's judged fairly and found wanting will work to improve. The condition is fairness and a visible path up. Judge someone against an irrelevant standard, or judge them with no way to climb, and the judgment just stings.

In the product. Vanity dashboards tell users they're using the product. Comparative benchmarks tell users where they actually stand, and the second kind changes behavior. HubSpot's benchmark dashboard shows your open rates and conversion next to "companies like yours" of similar size and industry, and suddenly the numbers mean something. A 24% open rate looks fine alone; sitting next to a 38% peer average, it looks fixable. The mechanism is social comparison, documented by Festinger back in 1954: we evaluate ourselves against others like us, and the product that shows that comparison is the product that drives the behavior.

The agent move. A benchmark dashboard shows the gap and leaves the user to figure out the climb. An agent can be the fair judge that also coaches the retake: it has seen this user's actual usage, can grade the thing they care about, "your setup is solid, your segmentation is where the money is leaking," explain the grade, and walk them up. Fair judgment with a path attached, per user.

WHAT WORKS

- Anonymize and aggregate the peer data. "Companies like yours" beats naming names.
- Show both the gap and the upside. "You're at 24%, top quartile is 38%, here's what they do differently."
- Make it weekly or monthly, not real-time. Daily benchmarks are noise.

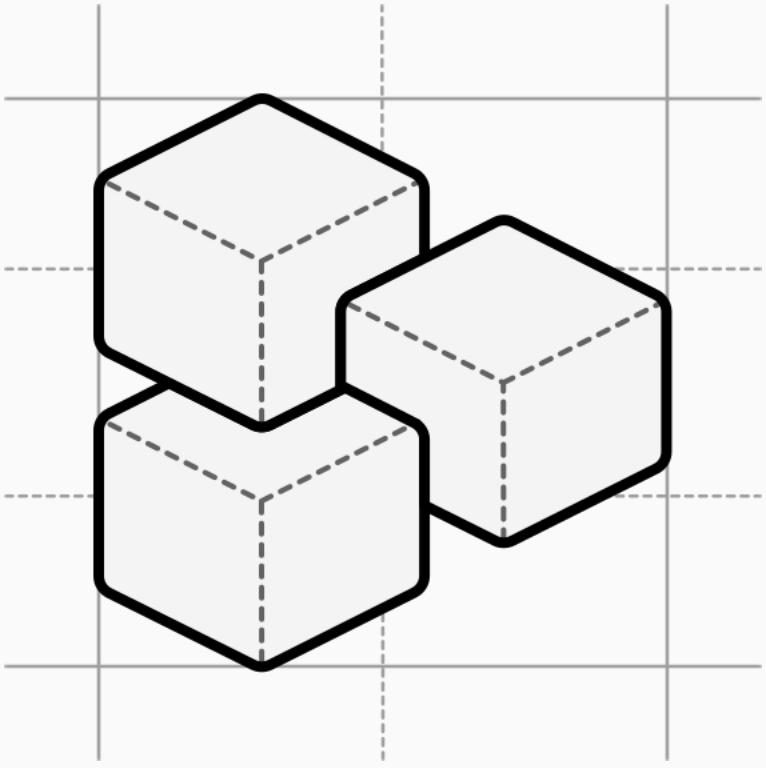
WATCH FOR

- Comparisons that shame. "You're in the bottom 10%" demotivates more than it moves.
- Irrelevant peer groups. A five-person startup against enterprise teams is discouraging, not useful.
- Showing the gap with no path to close it. A comparative number needs an action attached.

> **The shift. A usage metric is a mirror. A comparative metric is a lever. Most products show the mirror and call it analytics; the ones that change behavior show the lever.**

Expansion

Deeper use, longer life. Expansion is the part of the flow channel past the first plateau: the user has the basics, and now the product has to keep offering a next thing, or mastery turns into boredom and they leave. These patterns are about surfacing what's possible and making the value visible enough to grow into.



Expansion · Pattern

12

Hotspots and pulsing dots

THE RULE: THE INTERFACE IS A PROMISE ABOUT WHAT'S POSSIBLE

The principle. Hand someone a plastic guitar controller and nobody asks if they can stage-dive; the controller silently defines what's possible. Players infer the entire set of possible moves from the surface in front of them, not from a manual. The flip side is the trap: anything not visible on that surface effectively doesn't exist to the player. Users learn the shape of your product in the first two weeks, then their eyes stop scanning, and everything you ship after that sits there unseen.

In the product. Static interfaces go invisible. The fix is small, intentional motion: a pulsing dot on a new or underused feature, a subtle "New" badge, anything that breaks the visual habit people have built. Typeform pulses on builder features users haven't tried; Productboard pairs the dot with a one-sentence tooltip; Miro tags new tools with a badge in the toolbar. In each case the dot disappears the moment the user engages, so it doesn't become permanent noise. The mechanism is attentional capture: our eyes are pulled to movement and novelty, and one small pulse redraws the user's map of what the product can now do.

The agent move. A pulsing dot fires for everyone, including people who already use the feature. An agent can point attention only where it's earned: surface the new capability to the specific users whose work it would actually change, with a line on why it's relevant to them, and stay quiet for everyone else. Attention directed by usage, not sprayed across the whole base.

WHAT WORKS

- Two or three active hotspots at most. More and the product feels anxious.
- Pair the dot with one sentence of context. A bare dot just looks like a bug.
- Auto-dismiss after the user engages once. Forever-dots train people to ignore the system.

WATCH FOR

- Pulsing every feature at once. The signal collapses.
- Dots that fire for users who already use the feature.
- Animation loud enough to disrupt the rest of the UI. Subtle outperforms aggressive.

> **The shift. Feature adoption isn't a marketing problem, it's an attention problem. Most products have plenty of features and no way to direct attention to one. A pulsing dot solves more of that than another announcement email.**

13

Inline announcements and banners

THE RULE: THE CHANGE HAS TO BE VISIBLE WHERE IT HAPPENS

The principle. A basketball net adds nothing to the rules; it exists so the made shot is visible to everyone. Feedback has to appear where the action is, or it may as well not have happened. Announce a change somewhere the user isn't, and to them the change doesn't exist. The made basket in an empty gym still counts, but nobody cheers.

In the product. Most teams announce new features by email and are then surprised at low adoption, but the math is simple: SaaS email open rates sit around 20 to 25%, so three-quarters of your users never see the announcement. The version that works puts the news inside the product, in the surface where the feature actually lives. Notion drops "what's new" prompts in the relevant workspace area; Figma announces a new toolbar feature by tagging it in the toolbar itself. This mechanism isn't psychological, it's structural: you can't open-rate your way out of an email nobody opened, and in-product placement skips the delivery problem entirely. Tier it by importance, too: critical changes like AI being switched on get a banner or modal, important ones get a banner, nice-to-knows get a changelog entry and no interruption.

The agent move. A banner shows the same announcement to everyone and links to a generic changelog. An agent can deliver the news as feedback in context: tell the users who've touched the related feature, in the surface where they'd use the new thing, and link them straight to trying it, not to a post about it. The announcement lands where the action is, for the people the action is for.

WHAT WORKS

- Three tiers. Modal or persistent banner for critical, banner for important, silent changelog for the rest.
- Target by relevance. A banner about a new Lookups API should only fire for people who use Lookups.
- Set expiration dates. A banner that lives forever becomes chrome and stops being seen.

WATCH FOR

- Modal popups for everything. The modal is your highest-friction surface; save it for genuinely critical news.
- Banners that link to a generic changelog post instead of the feature itself.
- Sending the same thing by email and in-product. Pick the channel based on recent behavior.

> **The shift. An email is a request to go somewhere. An in-product banner meets the user where they already are. For adoption, the second one wins every time.**

Expansion · Pattern

14

The "What's New" widget

THE RULE: ALWAYS A NEXT THING, AND LET THEM PULL IT

The principle. A good game always has a next thing waiting and a visible sense of progress toward it, so you're never standing at a wall. But the strongest version hands the choice to the player: the next thing is there to pull when you want it, not pushed into your face. Interfaces feel best when the player's hands are on the controls, when the next move is theirs to take rather than forced. Pull beats push for anyone already engaged.

In the product. Banners interrupt, emails get deleted, a changelog at /changelog gets ignored. For your engaged users, the version that works is the in-product "what's new" widget: a bell icon with a notification badge, persistent, pull-based instead of push-based. Linear set the bar, with entries written user-first ("you can now do X"), a screenshot or GIF in each, links straight to docs, and a weekly-ish cadence that keeps the badge worth checking. Loom goes further and attaches a fifteen-second Loom to each entry, because for a visual feature a short demo beats a paragraph. The whole reason it outperforms banners and emails for non-critical news is the pull: people who care check it, people who don't aren't interrupted.

The agent move. A changelog widget still shows the same list to everyone. An agent can curate the user's "what's new" to what would actually change their work, lead with the one update that matters to them, and offer to set it up on the spot rather than linking to a doc. The next thing is always there to pull, and it's the right next thing for this person.

WHAT WORKS

- Categorize entries. New feature, improvement, fix, with a small icon so people orient fast.
- Write user-facing copy. "Signing in is now 40% faster," not "refactored authentication."
- Put media in every entry. Screenshot, GIF, or short video. Words alone underperform.

WATCH FOR

- Posting once a quarter. The badge needs fresh fuel, and long gaps train people to ignore it.
- Marketing-grade language. "Reimagining the future of work" reads as noise; "fixed the export crash" reads as a real product.
- Burying the widget in a settings menu. If it takes more than five seconds to find, it isn't pull-based, it's invisible.

> **The shift. Emails go to users who might come back. A changelog widget meets users who already came back. Different jobs, different surfaces.**

Expansion · Pattern

15

Feature suggestions based on usage

THE RULE: SERVE THE REQUEST AND STEER ONE LEVEL UP

The principle. This is the centerpiece move from game design, called collusion. An old theme-park attraction had a problem: free-roaming players got bored in two minutes. The fix was that the enemy ships stopped acting purely in their own interest; they'd attack, then flee toward an island the player hadn't seen yet, timed so the next interesting thing came into view at exactly the right moment. The characters held two goals at once: their own, plus quietly steering the player toward the best possible experience. Players reported total freedom. The job isn't to answer the request and stop, it's to answer the request and steer the user one level up.

In the product. Most products surface advanced features the way a 1990s homepage did, one big banner everyone sees regardless of what they actually do. The pattern that works is contextual suggestion: the right next feature at the right moment, based on what the user just did. Grammarly does it well, "this sentence has advanced clarity issues, upgrade to see suggestions," landing the pitch at the exact moment the user feels the limit. The mechanism is the mere exposure effect, where repeated relevant encounters make a user likelier to try the feature, but relevance is the unlock; repetition without relevance is just noise.

The agent move. This is collusion, and the agent is the first onboarding surface that can actually run it. It answers the thing the user asked, then flees toward the island: "done; by the way, the thing you just did by hand, I can watch for and do automatically, want that?" Served the request, steered them one level up, and the user comes away feeling they discovered the next feature themselves.

WHAT WORKS

- Trigger after a relevant action. "You just built your fifth report; there's a way to automate this."
- Quantify the upside. "Saves two hours a week" beats "schedule recurring reports."
- Make it dismissible, and don't re-show the same suggestion for a couple of weeks.

WATCH FOR

- Generic upsell modals shown to everyone. It reads as a banner ad inside your own product.
- Suggestions that fire too early. A brand-new user doesn't need the advanced feature yet.
- Stacking three suggestions at once. The brain picks zero.

> **The shift.** Feature discovery is a recommendation problem, not a marketing problem. Most teams build the feature, write a help article, and wonder why nobody finds it. The fix lives inside the product, fired by real usage.

16

Win-back and re-engagement flows

THE RULE: THE VALUE YOU'VE BUILT IS THE PULL BACK

The principle. In one game, collected yarn balls awarded points that did nothing, and players quit collecting within minutes; in another, the rings you collect protect you and buy extra lives, and players chase every one. The difference is whether the thing you've accumulated is genuinely worth something inside the system. That stored-up value is also what pulls a lapsing player back: the more real value you've built, the more it costs to walk away, and loss aversion does the rest.

In the product. Most products treat churn as an event, the day someone cancels. It's really a process that starts weeks earlier, when the user quietly stops logging in. The good products catch it early and pull people back before they've consciously decided to leave. Dropbox frames it as "your files miss you," with the specific count attached, "847 files and 23 shared folders waiting." The mechanism is loss aversion, the same force that makes people hold a losing stock: the more the user has built inside the product, data, settings, integrations, history, the stronger the pull. Worth keeping in view, acquiring a customer costs five to twenty-five times more than keeping one, and most teams spend almost nothing on disengagement.

The agent move. A "we miss you" email is generic and the user knows it. An agent can make the stored value specific and personal, deep-link them back to the exact project they left, not the homepage, and name what they'd be walking away from, "you'd built 47 hours of work in here." It's the rings made visible at the moment leaving is on the table.

WHAT WORKS

- Define the at-risk signals. No login in seven days, key feature unused for fourteen, session length halved. Then trigger a three-touch sequence.
- Touch one is an in-app banner on return: "welcome back, want to pick up where you left off?"
- Touches two and three are emails a week apart, escalating from helpful to direct, deep-linked to their last project.

WATCH FOR

- Generic "we miss you" with no specific value reminder. It reads as marketing automation, not a product.
- Quantifying vanity. "You logged in 30 times" isn't value.
- Firing the win-back at day 60 instead of day 7. By then they've already moved on.

> **The shift.** Retention isn't a metric to measure, it's a flow to design. Most teams measure churn; the ones that move the number build a specific sequence around the signals that come before it.

Expansion · Pattern

17

Usage milestones and progress celebrations

THE RULE: EXPERIENCES ARE THE ONLY THING THAT CHANGE PEOPLE

The principle. Games create experiences, and experiences are the only thing that can actually change a person. The real measure of a game isn't whether you learned its menus, it's whether it changed how you behave. There's a companion rule about expression: people who can see and show what they've made feel proud, and pride is the thing they share. Put together: the win isn't that the user configured your product, it's that they work differently now because of it, and they can feel and show the proof.

In the product. The longer a user stays, the easier they are to keep, but only if they can feel the value they've piled up. Most products stack that value behind the scenes, hours saved, reports generated, deals tracked, and never show it. The fix is explicit milestones surfaced at intervals. Chameleon celebrates after a user closes 50 boards; Grammarly's weekly writing insights go viral because users screenshot and share them. The mechanism is sunk cost used in the user's favor: the more value they've visibly created, the harder it is to leave, because they're not abandoning a tool, they're abandoning 47 hours of their own work. There's a second payoff inside companies: the milestone becomes the champion's internal evidence, the defensible budget line at renewal time.

The agent move. An agent can narrate the transformation as it happens, which is what makes it real to the user: "three weeks ago this report took you an hour; today you asked one question." That single sentence is retention, the justification for an upgrade, and the advocate's story to retell, all generated from usage data the user would never have assembled themselves.

WHAT WORKS

- Celebrate value, not activity. "Saved 47 hours" beats "logged in 47 times."
- Use both in-app and email. Email travels, the screenshot in Slack, and in-app reinforces session to session.
- Tie the milestone to expansion when it's natural. "You've hit 1,000 contacts; the Growth plan is unlimited" lands at the right moment.

WATCH FOR

- Vanity milestones. "30 days in a row" is a streak, not value.
- Daily celebrations. Reward fatigue sets in fast; weekly or milestone-based is the cadence.
- A milestone with no next step. Either the good feeling ends there, or you give them something to do with it.

> **The shift. Every product has value the user is creating, and most forget to tell the user about it. The ones that show it, keep people.**

The dual use

Run this with your favorite LLM

THE FILE IS THE EXPERT; YOUR AI ASSISTANT DOES THE LOCALIZING

These patterns work two ways. You can read them and build, the same as any swipe file. Or you can hand this file to your favorite LLM and have it do the heavy lifting, which is how I'd actually use it. The PDF is the expert material; your LLM is the thing that localizes it to your product, because onboarding advice is always specific to one product and yours is the only one your LLM can see in context. The prompt is not the product. This file is.

ASK YOUR LLM	WHAT YOU GET BACK
"Here's my product and where users drop off. Which of these 17 patterns fit, and which two should I ship first?"	A shortlist tied to your actual leak, not all seventeen.
"Design pattern 1, aha moment shortcuts, for my product. What's the one moment, and how do I get a user there in the first session?"	A concrete first-session plan, named moment, named steps.
"Write the contextual tooltips for my three highest-value features, at the moment each one is first used."	Draft microcopy you can paste in and edit.
"Take pattern 7, behavior-based triggers, and map my five highest-value events to one message each."	Your trigger map, ready to build.
"I'm an agent-led product. For each pattern I pick, what's the agent move, and what would the agent actually say?"	The agent-led version, in your agent's voice.

The move that worked for me: pick your stage, paste in the two or three patterns for it, give it real context about your product and your numbers, and ask it to design the thing and draft the copy. Then you edit. You're not starting from a blank page, you're editing a draft built on rules that have held up for fifty years.

Seventeen patterns, one surface. When the tour, the nudge, the empty state, and the milestone all live in one agent, the product does the work of figuring out which one the user needs, instead of the user.

The Onboarding Loop Protocols

BAL SIEBER · LEVEL DESIGN ADD-ON